WEEKEND ASSIGNMENT • PROJECT 1 OF 5

FAQ Mini Chatbot

Build a rule-based chatbot that answers questions from a small FAQ dataset using keyword and fuzzy matching.

DURATION	DIFFICULTY	STACK	DOMAIN
1 weekend	**Beginner**	**Python, Streamlit**	**NLP / Chatbots**

Project Overview

You will build a small chatbot that answers user questions by matching them against a predefined FAQ dataset. There is no machine-learning model to train here — the intelligence comes from text normalisation and fuzzy string matching. The goal is to design it cleanly enough that an LLM or ML model could be swapped in later without rewriting the interface.

You can pick any FAQ topic you like: HR policies, a college admissions FAQ, a product help page, or banking FAQs. Prepare **at least 25 question–answer pairs** in a JSON or CSV file.

Learning Objectives

- Text preprocessing: lowercasing, punctuation removal, tokenisation, stopword removal.
- String similarity: fuzzy matching with `rapidfuzz` (or `difflib`) and/or TF-IDF cosine similarity with scikit-learn.
- Separating the matching engine from the user interface (clean module boundaries).
- Building a simple chat UI with Streamlit, or a clean CLI loop.
- Full Git workflow: branches, PRs, Issues, releases.

Functional Requirements

1. **FAQ data file:** `data/faqs.json` (or .csv) with at least 25 Q&A pairs, each with a question, an answer, and optionally alternate phrasings.
2. **Matching engine** (`src/matcher.py`): a function `get_answer(user_question) -> (answer, confidence)` that normalises the input, scores it against every FAQ question, and returns the best match with a similarity score between 0 and 1.
3. **Confidence threshold:** if the best score is below a threshold (e.g. 0.55), reply with a polite fallback such as “I’m not sure about that — here are some questions I can answer” and show the 3 closest questions.
4. **Interface:** EITHER a Streamlit chat page (using `st.chat_input` / `st.chat_message`) OR a CLI loop that keeps asking until the user types `quit`. Streamlit is preferred.
5. **Chat history:** the UI should display the running conversation, not just the last answer.
6. **Edge cases:** empty input, very long input, and gibberish must not crash the app.
7. **Bonus (optional):** log unanswered questions to a file so the FAQ can be improved; show the confidence score next to each answer.

Required Repository Structure

```
faq-chatbot/
|-- README.md
|-- requirements.txt
|-- .gitignore
|-- data/
|   `-- faqs.json            # your 25+ Q&A pairs
|-- src/
|   |-- __init__.py
|   |-- preprocess.py        # text cleaning helpers
|   `-- matcher.py           # get_answer() lives here
|-- app.py                   # Streamlit UI (or cli.py for CLI)
`-- tests/
    `-- test_matcher.py      # at least 3 simple asserts
```

Keeping `matcher.py` free of any UI code is part of the evaluation — the same engine must work from both a test file and the app.

Suggested Technical Approach

1. Write `preprocess.py` first: a `clean(text)` function (lowercase, strip punctuation, collapse whitespace).
2. Implement matching v1 with `rapidfuzz.fuzz.token_set_ratio` — score the cleaned user input against every cleaned FAQ question and take the max.
3. Optionally upgrade to TF-IDF: vectorise all FAQ questions with `TfidfVectorizer`, transform the user input, and use cosine similarity. Compare both in your README.
4. Wire the engine into Streamlit last. Keep all session state in `st.session_state["messages"]`.
5. Write three tests: an exact question returns the right answer; a paraphrased question still matches; gibberish falls below the threshold.

Install & run

```
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install streamlit rapidfuzz scikit-learn pytest
pip freeze > requirements.txt
streamlit run app.py
pytest tests/
```

Project-Specific Git Notes

- Suggested branches: `feature/matching-engine` and `feature/streamlit-ui`.
- Suggested Issues: “Create FAQ dataset”, “Build matcher with fuzzy scoring”, “Add Streamlit chat UI”, “Add fallback for low confidence”.
- Commit the dataset — it is small and hand-made, so it belongs in the repo (unlike large downloaded datasets in other projects).

Suggested Weekend Timeline

You have roughly two and a half days. This split keeps the Git workflow alive throughout the weekend instead of being an afterthought.

When	What to do
Friday evening	Read this document fully. Create the repo, .gitignore, README skeleton, folder structure, and your 3+ GitHub Issues. First commit and push.
Saturday morning	Set up the environment, get the data/inputs, and build the first core module on a feature branch. Open and merge your first PR.
Saturday evening	Build the second core module (model / matching / dashboard logic) on a new branch. Commit in small steps.
Sunday morning	Finish the user-facing part (CLI / UI), handle edge cases, merge remaining PRs.
Sunday evening	Polish the README, add screenshots, record the demo, tag v1.0, publish the Release, and submit.

Git & GitHub Requirements (Mandatory)

This assignment is primarily an evaluation of how you use Git and GitHub. A working project with a poor commit history will score lower than a slightly imperfect project with a clean, professional workflow. Follow every requirement below.

1. **Repository setup:** Create a new GitHub repository (public or private as instructed by your mentor). Initialise it locally with `git init`, add the remote, and make your first commit the project skeleton (README + .gitignore + folder structure).
2. **Commit discipline:** Make a minimum of **10–15 atomic commits**. Each commit should do one thing. Use conventional commit messages, e.g. `feat: add TF-IDF vectorizer`, `fix: handle empty input`, `docs: add setup instructions`. A single end-of-weekend dump commit is an automatic deduction.
3. **Branching:** After the initial setup commit, **never commit directly to main**. Create at least **2 feature branches** (named like `feature/preprocessing`) and merge each into main through a Pull Request on GitHub.
4. **Issues:** Before you start coding, create at least **3 GitHub Issues** describing the tasks you plan to do (e.g. “Build data loading module”). Reference them in your PRs with “Closes #2” so they close automatically on merge.
5. **Pull Requests:** Each PR must have a meaningful title and a short description of what changed and why. Review your own diff in the GitHub UI before merging — this is a habit, build it now.
6. **.gitignore:** Your repository must never contain virtual environments, datasets, model binaries, IDE folders, or `__pycache__`. Add a proper Python .gitignore in your very first commit.
7. **Release:** When finished, tag your final commit as `v1.0` and publish a GitHub Release with short release notes (what works, known limitations).

Command reference

```
# one-time setup
git init
git remote add origin https://github.com/<you>/<repo>.git
git add . && git commit -m "chore: initial project skeleton"
git push -u origin main

# per feature
git checkout -b feature/<name>
# ...work, then commit in small steps...
git add <files> && git commit -m "feat: <what you did>"
git push -u origin feature/<name>
# open a Pull Request on GitHub, link the Issue, merge, then:
git checkout main && git pull

# at the end
git tag -a v1.0 -m "First working version"
git push origin v1.0
```

README Requirements

Your README.md is the front door of the repository. It must contain, in this order:

1. **Project title and one-line description** of what the project does.
2. **Problem statement** — 2–3 sentences on what problem this solves.
3. **Setup instructions** — exact commands to create a virtual environment, install dependencies from requirements.txt, and run the project. Assume the reader knows nothing.
4. **Usage examples** — sample input and the output it produces (text or screenshot).
5. **Screenshots / demo** — at least one screenshot of the working project.
6. **Approach** — a short paragraph on how it works (the techniques/libraries used and why).
7. **Known limitations** — honesty is rewarded; list what does not work well yet.

Evaluation Rubric

Criterion	Weight	What we look at
Git & GitHub workflow	40%	Commit quality and atomicity, branch usage, PRs with descriptions, Issues linked and closed, v1.0 release, clean .gitignore.
Code quality & structure	25%	Follows the required folder structure, readable functions, no hard-coded paths, requirements.txt is complete and reproducible.
README & documentation	20%	All README sections present, setup instructions actually work on a fresh machine, screenshots included.
Working output	15%	The project runs end-to-end and produces sensible results as described in the requirements.

Submission Checklist

Go through this list before you submit. Every unchecked box costs marks.

[ ] GitHub repository link shared with your mentor (add them as a collaborator if private).

[ ] Commit history shows 10+ atomic commits with conventional messages.

[ ] At least 2 feature branches merged into main via Pull Requests.

- [ ] At least 3 Issues created and closed via PR references.
- [ ] v1.0 tag and GitHub Release published with release notes.
- [ ] README is complete per the README checklist in this document.
- [ ] requirements.txt installs cleanly in a fresh virtual environment.
- [ ] No datasets, virtual environments, or model binaries committed to the repo.
- [ ] A short demo video (1–2 minutes screen recording) or 3–4 screenshots attached to the Release or linked in the README.

Rules & Use of AI Tools

- You may use AI assistants and online resources to learn, but you must be able to explain every line of code in your repo. A short walkthrough/viva may follow submission.
- Do not copy a complete project from GitHub or a tutorial. We check commit timestamps and history — a copied project is obvious and will be treated as a failed assignment.
- Work individually. Asking peers conceptual questions is fine; sharing code is not.
- If you get blocked for more than an hour, message your mentor with what you tried — asking good questions is part of the evaluation, not a penalty.